Introduction to class and object

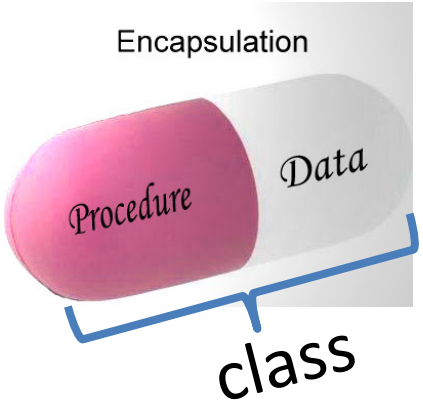
Sections 9.1 and 9.2

Define individual methods vs. class

- Both can increase code reusability.
- Defining functions alongside the main function is like expanding a company, but only to the level of consultant service.
  - Given a request from a user, provide service to satisfy that request (for example, given an integer, find out whether it is prime or not).
- To create a product, define a class which combines **attributes** (properties describing products) and **operations** (methods) performed on the attributes.

class = data + operations on data

```
class ThanksgivingDinner {
    public: //operations
        cookTurkey();
        cookPie();
    private: //data member
        Turkey peace;
        Pumpkin cuty;
};//do not forget ;
```

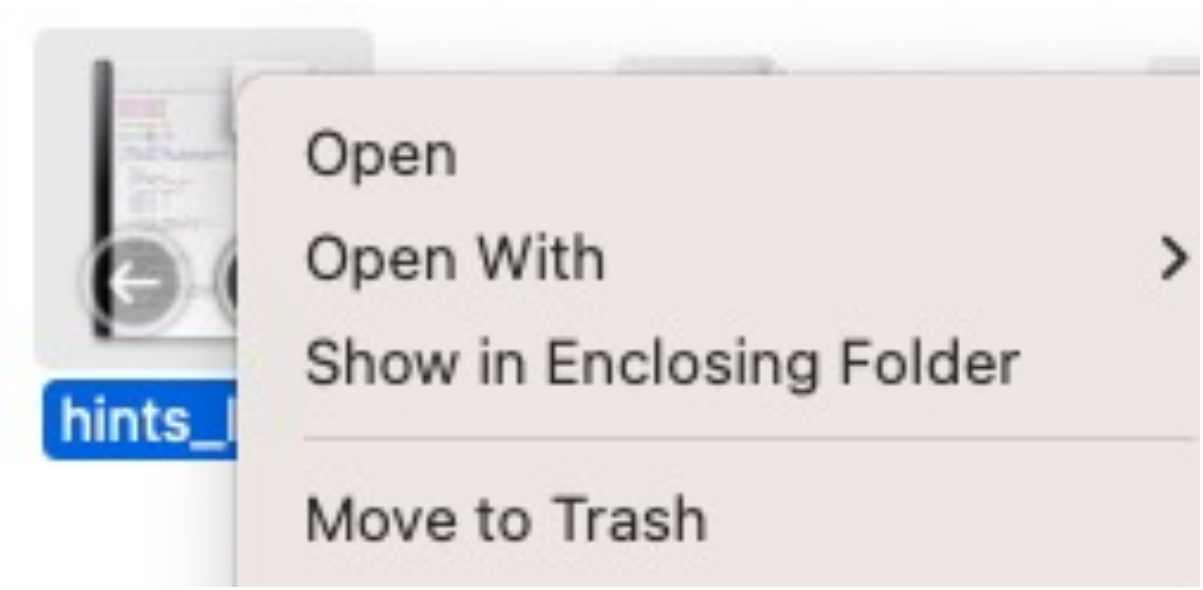


Hamburger class

an object (aka instance) of Hamburger

Not every operation fits for an object

- Keep data secure and valid at all stages of an object's life
- The only way to access or modify data is through the provided operations



Example of encapsulation

- Given a bank account, the data can be account name, pin, account number, and balance.
- Operations can be log in (using your name and pin), balance inquiry, deposit and withdrawal.
- ATM machine provides buttons for operations.

Why Encapsulation?

- attributes: properties shared by all objects of the same class
- Different classes have different attributes.
- A person object has height attribute, and operations on this attribute can include
  - initialize (height-at-birth)
  - change (increase when young and shrink when one gets old)
  - get height

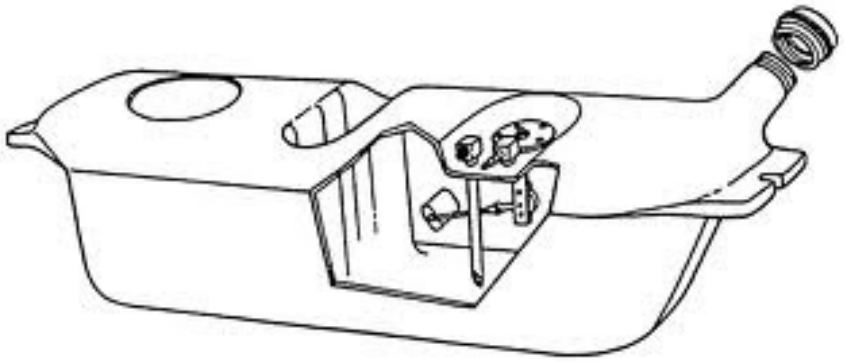
Why Encapsulation? II

- A car has attribute current gas level in a fuel tank. Corresponding operations can be
  - Initialize current gas level
  - Increase gas level by pumping more gas or decrease gas level by driving
  - Get the current gas level
- Different operations on different attributes.
  - Can you ask MPG (Miles Per Gallon) of a person?
  - Can you ask BMI (Body Mass Index) of a car?

Class and Object

A class is the blueprint from which objects are made.

- For example, the class serves as the blueprint for a car's fuel tank, while objects are the actual instantiated fuel tanks.
- A class is a type (like int), and an object is a variable (like x)



Data member vs. methods

- Things object knows are data members
- Things object does are methods
  - Object
    - fuel tank
  - Data members
    - Current gas level
  - Methods
    - Fill tank with more gas
    - Send gas through the gas lines
    - Get current gas level of tank

Encapsulation

- Combine data members and methods in one package

current gas level
Fill tank with gas
Send gas through gas line
Get current level of tank

- Encapsulation is closely related to information hiding
  - encapsulation hides the implementation details from the user of the object.
  - One can use a fuel tank without knowing how it works.

- Do you need to be an auto engineer to operate the car?
- Do you need to be a system programmer to use string class?

Declare class FuelTank

```
#include <iostream>
class FuelTank { //partial code link in onlinegdb
public:
   //function members (aka operations)
   //1. Constructor(s): initialize data member
   //2. fill gas to the fuel tank
   //3. send gas from tank to gas line
   //4. read the current gas level
   //5. destructor:
   //delete any dynamically allocated memory spaces.
private:
   int currGasLevel;
}; //do not forget ;
```

- This is the code of FuelTank.hpp.
- Implementation of constructors and methods declared in FuelTank.hpp is in FuelTank.cpp.

Access Specifiers

- **private:** only members in the same class can access
- **public:** other classes or functions outside the class can access
- **In class, private by default:**
  - if access specifiers like public: or private: are omitted, C++ defaults to private.
  - This reinforces the "information hiding" concept.

Take home message – what is class?

- class is the encapsulation of data members and methods that operate on those data members.
- Next task:
  - Define class FuelTank: ie, implement FuelTank.cpp
  - Construct an object tank from class FuelTank
  - Use object tank

Define class FuelTank in FuelTank.cpp

```
//Every class has a default constructor.
//A constructor is a method with the same
//name as the class, but no return type –
//not even void, because constructors never
return a value
//Default constructor has no parameter.
FuelTank::FuelTank() {
    currGasLevel = 0;
}
```

- Symbol :: is scope operator.
- FuelTank:: before FuelTank means FuelTank belongs to FuelTank class.
- Similarly, std::string means string belongs to namespace std.

1. The first letter of class name is capital.
2. Data members can be accessed by every method/constructor of the same class directly.
3. So, there is no need to pass data member as parameters in those methods/constructors.

Common Error: declare data member again in constructor

```
FuelTank::FuelTank() {
  int currGasLevel;
  currGasLevel = 0;
}
```

If you declare currGasLevel inside a constructor, then currGasLevel is a **local variable** of the constructor, it has nothing to do with data member `currGasLevel`.

Optional: **Explicit Member Initializer Lists in Constructors**

- In previous slide, initialize data member inside the constructor body

```
FuelTank::FuelTank() {
    currGasLevel = 0;
}
```

- **Member Initializer Lists** is considered a "best practice" in modern C++.

FuelTank::FuelTank() : currGasLevel(0) {}

Define class FuelTank: II

```
//Non-default constructor has at
//least one parameter.
FuelTank::FuelTank(int startLevel) {
    currGasLevel = startLevel;
}
//For simplicity, assume that
//startLevel is valid, non-negative and
//no larger than capacity of fuel tank.
```

Default Constructor vs. Non-Default Constructors: public defender analog

- If you *only* write a non-default constructor, C++ will no longer provide the default one for you automatically.
- If you cannot afford an attorney, one will be appointed for you.

Default Constructor vs. Non-Default Constructors: public defender analog II

Scenario	In the Courtroom	In C++ Programming
Doing nothing	You show up without a lawyer. The court provides a **Public Defender** automatically.	You write a class without any constructors. C++ provides a **Default Constructor** automatically.
Taking Action	You hire your own **Private Attorney**	You write your own **Parameterized (non-default) Constructor**.
The Result	Because you have representation, the court **will not** give you a public defender.	Because you defined a constructor, C++ **will not** generate the default one.

Define class FuelTank: IV

```
//Fill gas to the fuel tank
//For simplicity, we do not consider
//capacity of a fuel tank.
void FuelTank::fillGas(int gallons) {
    currGasLevel += gallons;
}
```

Define class FuelTank: V

```
//send gas from tank to gas line
void FuelTank::sendGas(int gallons) {
    if (currGasLevel >= gallons) {
        currGasLevel = currGasLevel -
                       gallons;
        //same as
        //currGasLevel -= gallons;
    }
}
```

Define class FuelTank: VI

```
//Getting the current gas level
//is an accessor (or getter).
//add const means the function can
//read but not modify data member.
int FuelTank::getGasLevel() const {
    return currGasLevel;
}
```

Use class FuelTank

- Create an empty fuel tank.
- Add 5 gallons of gas to it.
- Find out how much gas in the tank. Print out the result to the screen.
- Send 1 gallon of gas to gas line.
- Find out how much gas in the tank. Print out the result to the screen.

Use FuelTank class

```
int main() {
    //Create an empty fuel tank.
    //call default constructor
    //of FuelTank class.
    FuelTank tank;
```

Related:
//instantiate an empty string object str
//string str;

Use FuelTank class: II

```
//Add 5 gallons of gas to the tank.
tank.fillGas(5);

//Find out how much gas in the tank.
int tankLevel = tank.getGasLevel();
//Print out the result to the screen.
cout << "The tank has " << tankLevel
     << " gallons of gas.";
```

```
The tank has 5 gallons of gas.
```

Use FuelTank class: III

```
//Send 1 gallon of gas to gas line.
tank.sendGas(1);

//Find out how much gas in the tank.
tankLevel = tank.getGasLevel();
//Print out the result to the screen.
cout << "The tank has " << tankLevel
     << " gallons of gas.";
return 0;
}
```

```
The tank has 4 gallons of gas.
```

Summary: class and object

Core Definition

- **Class:** A **blueprint** or template (a type, like int).
- **Object:** An **instance** of a class (a variable, like x).
- **Structure:** A class combines **Attributes** (what it knows/data) and **Operations** (what it does/methods).

Summary: Encapsulation and Information Hiding; Access Control

- **Encapsulation:** Packaging data and methods together
- **Information Hiding:** Hiding internal implementation details from the user.
- **Access Control:** Use public: for operations users need and private: to keep data secure and valid

Summary: Implementation

- **Semicolon:** Always end a class declaration with a ;
- **Scope Operator (::):** Used in the .cpp file to show a method belongs to a specific class (e.g., FuelTank::fillGas)
- **Constructors:** with the **same name as the class** and **no return type**. They initialize data members
  - *Default:* No parameters
  - *Non-default:* At least one parameter

Summary: Implementation II

- **Getters (Accessors):** Methods that read data. Use const at the end to ensure the method doesn't modify the object
- **Setters (Modifiers):** Methods that change the value of data members

Quick Quiz

1. Based on the "blueprint" analogy, if "House" is the class, what is an "Object"?

A. The architectural drawing

B. The materials used to build it

C. A specific house at 123 Main Street

2. Which of the following is NOT a characteristic of a C++ Constructor?

A. It has the same name as the class.

B. It must return void.

C. It is used to initialize data members.

Quick Quiz: II

3. True or False: If you write a non-default constructor, C++ will still provide a default one for you automatically.

4. Why put const at the end of the getGasLevel() method?

A. To make the gas level constant forever.

B. To prevent the method from accidentally modifying any data members.

C. To speed up the code execution.

Quick Quiz: III

5. What is the correct operator used to define a class method outside of the class declaration (in the .cpp file)?

A. ->

B. .

C. ::

Quick Quiz: IV

6. For string class, which one is constructor, getters, or setters? link for string class declaration

A. string();

B. string (const char* s);

C. size_t size() const;

D. char& operator[] (size_t pos);const

E. char& back();const

F. string& operator+= (const string& str);